

Constrained Pseudorandom Functions

Revisited

Soutenance de projet — Master 1 Cryptologie

Julian Mouthon & Mario Razafinony

Sorbonne Université — 23 mai 2026

Plan de la présentation

- 1 Motivations et définitions
- 2 Construction BMO17
- 3 L'attaque CJ25
- 4 Alternatives à RSA
- 5 CPRF hachée et sécurité
- 6 Résultats expérimentaux
- 7 Conclusion

Pourquoi les fonctions pseudo-aléatoires contraintes ?

Problème concret

- On délègue parfois des capacités d'évaluation
- Mais on ne veut pas révéler la clé secrète complète
- Exemple : *Searchable Symmetric Encryption*, stockage externalisé

Idée clé

- Une **PRF** produit des sorties indiscernables du hasard
- Une **CPRF** restreint l'évaluation à un sous-ensemble d'entrées
- Sans révéler la clé maître

Analogie

Comme un **badge d'accès partiel** : il ouvre certaines portes, pas toutes, sans révéler le passe maître.

Notre contrainte principale

$$C_n(x) = [x < n]$$

Autoriser les entrées strictement inférieures à n .

Définition formelle d'une CPRF

Une CPRF est donnée par quatre algorithmes :

$\text{Setup}(1^\lambda) \rightarrow K$

Génère la clé maître K

$\text{Eval}(K, x) \rightarrow y$

Évalue avec la clé maître

$\text{Constrain}(K, C) \rightarrow K_C$

Dérive une clé contrainte pour la contrainte C

$\text{EvalC}(K_C, x) \rightarrow y$

Évalue avec la clé contrainte (si $C(x) = 1$)

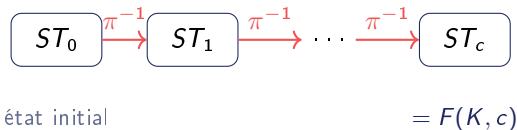
Propriété de correction

$$\forall x \text{ tel que } C(x) = 1 : \quad \text{EvalC}(K_C, x) = \text{Eval}(K, x)$$

La construction de Bost, Minaud et Ohrimenko (2017)

Principe

- Basée sur une **permutation à trappe** π (RSA)
- On itère π^{-1} à partir d'un état initial secret ST_0

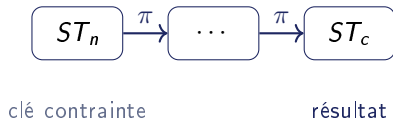


Clé maître : (ST_0, SK)

$$\text{Eval}((SK, ST_0), c) = \pi_{SK}^{-c}(ST_0)$$

Clé contrainte pour $C(x) = [x < n]$:

- Clé publique PK , état $ST_n = \pi^{-n}(ST_0)$, seuil n
- SK n'est **jamais** incluse



Implémentation RSA

Permutation directe (publique)

$$x \mapsto x^e \bmod N$$

Permutation inverse (secrète)

$$x \mapsto x^d \bmod N$$

avec $x^{ed} = x \bmod N$

Détails d'implémentation

- Bibliothèque OpenSSL (BIGNUM)
- Clés RSA de **4096 bits**
- Aléa sécurisé via `/dev/urandom`
- État initial ST_0 : entier aléatoire 256 bits

Propriété essentielle

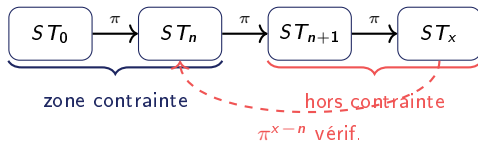
RSA est une **permutation** (bijection) sur $\mathbb{Z}_N^*$ — chaque image a une unique pré-image, ce qui permet l'itération cohérente de π^{-1} .

L'attaque de Cheng et Jaeger (2025)

Idée de l'attaque

L'attaquant exploite la **structure algébrique** de la CPRF :

- 1 Demande une clé contrainte (PK, ST_n, n)
- 2 Interroge l'oracle sur $x > n$, obtient ST_x
- 3 Calcule $\pi_{PK}^{x-n}(ST_x)$ et compare à ST_n
- 4 Si $ST_a = ST_n \rightarrow$ c'est une CPRF !



Avantage de l'attaquant

$$\Pr[\mathcal{A} \text{ gagne}] \approx 1 - \frac{1}{N}$$

Négligeable seulement si N est exponentiel.

Conclusion

BMO17 **n'est pas** IND-sécurisée sans hachage.

Architecture client–serveur TCP

- **Serveur** = oracle du jeu de sécurité
 - $\text{EVAL}(x)$: retourne $F(K, x)$ ou valeur aléatoire
 - $\text{CONSTRAIN}(n)$: retourne (PK, ST_n, n)
- **Client** = attaquant
 - Choisit n , demande la clé contrainte
 - Effectue des requêtes $x > n$
 - Vérifie la cohérence par π^{x-n}

Résultats

- **100 requêtes** par variante
- Version non hachée : succès ≈ 1 dès la 1^{ère} requête
- Version hachée : taux ≈ 0.5 (aléatoire)

Conclusion

L'attaque est **dévastatrice** sur la version non hachée. Le hachage neutralise complètement l'attaque.

Étude d'alternatives à la permutation RSA

Nous avons étudié trois alternatives :

F^{WEAK} — linéaire

$$\text{Eval}(S, \vec{x}) = S \cdot \vec{x}$$

Cassée : système linéaire soluble avec $N - 1$ requêtes contraintes.

Rabin

$$f(x) = x^2 \bmod n$$

Non injective : 4 pré-images impossible. : 4 pré-images pour chaque image. Impossible d'itérer correctement.

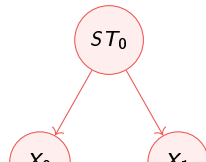
AES

$$f(x) = E_k(x)$$

Non adapté : pas de trappe publique.

Propriétés requises pour la construction BMO17

- 1 **Bijektivité** : une seule pré-image



F^{WEAK} : la construction linéaire

Construction

- Clé maître : matrice $S \in (\mathbb{Z}/p\mathbb{Z})^{M \times N}$
- $\text{Eval}(S, \vec{x}) = S \cdot \vec{x}$
- Contrainte $\vec{y}$: entrées orthogonales à $\vec{y}$
- Clé contrainte : $S_{\vec{y}} = S + \vec{d} \cdot \vec{y}^T$

Attaque par système linéaire

- Choisir $\vec{y} = (0, \dots, 0, 1)$
- Envoyer $N - 1$ vecteurs $\vec{x}_j \perp \vec{y}$
- Résoudre $X \cdot \vec{S}_i^T = \vec{k}_i$
- Retrouver S avec 1 requête d'évaluation

F^{WEAK} est cassée

Dès la **première évaluation**, l'attaquant retrouve S .

Renforcement par hachage

$$\text{Eval}_H(\vec{x}) = H(S \cdot \vec{x})$$

Retrouver $S \Rightarrow$ pré-image de $H \rightarrow$
infaisable.

Limite

F^{WEAK} n'est **pas** IND^* -NC sécurisée
 $\Rightarrow$ théorème CJ25 inapplicable
directement.

Renforcement par hachage — Construction CJ25

Principe

Appliquer un oracle aléatoire P sur la sortie de la CPRF :

$$\text{Eval}_H(K, x) = P(\text{Eval}(K, x))$$

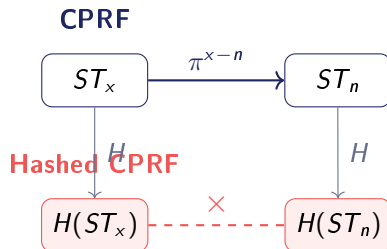
Lazy Sampling

P mémorise les paires (x, y) au fur et à mesure :

- Si $x \notin \sigma_P$: choisir $y \leftarrow \{0, 1\}^\lambda$
- Sinon : récupérer y mémorisé

Théorème (CJ25)

Si CF est $\text{IND}^*\text{-NC-CPRF}$ sécurisée et l'oracle est non-programmable, alors $\text{Hashed}[CF]$ est



Effet

L'attaquant ne peut plus **comparer** via les permutations publiques — le hachage brise la structure exploitable.

Configuration matérielle

Composant	Détail
Machine	Dell Inspiron 3501
CPU	Intel Core i7-1165G7
Fréquence	2.80 GHz ($\uparrow$ 4.70)
RAM	7.5 GB
OS	Ubuntu 24.04.4 LTS
Architecture	x86_64

Protocole

- 100 requêtes d'évaluation par variante
- Taux de succès de l'attaque CJ25 mesuré

Résultats obtenus

- **CPRF non hachée** : succès ≈ 1 dès la 1^{ère} requête — attaque totalement efficace
- **CPRF hachée SHA-256** : taux ≈ 0.5 — décision aléatoire, attaque inefficace
- **Hashed + lazy sampling** : taux ≈ 0.5 — résultats identiques à SHA-256
- F^{WEAK} : avantage significatif visible dès les premières requêtes

Observation clé

SHA-256 et lazy sampling donnent des résultats équivalents en pratique

Conclusion

Ce que nous avons fait

- Étudié et implémenté la CPRF de BMO17 (RSA, 4096 bits)
- Simulé l'attaque CJ25 en client–serveur TCP
- Évalué trois alternatives : F^{WEAK} , Rabin, AES
- Implémenté deux variantes hachées (SHA-256 et lazy sampling)
- Étudié la sécurité de F^{WEAK} hachée

Résultats

- Le hachage **neutralise** l'attaque CJ25
- RSA : seule permutation à trappe adaptée
- F^{WEAK} non IND*-NC $\rightarrow$ théorème CJ25 inapplicable

Perspectives

- Contraintes plus expressives (intervalles, circuits)
- Autres bases : hypothèse LWE, couplages
- Preuve formelle de Hashed[F^{WEAK}]

github.com/julian-mtn/bmo17-cprf